

# Relatório de Evolução do Projeto — GoodWe EV ChargeOps Chatbot

## Sprint 03 — Refactory Conversacional em LangChain

**Grupo:** André Fujinaga (RM569158) · Arthur Machado (RM569919) · Conrado Gracie (RM569157) · Guilherme Belo (RM570079) · Renato Sandreschi (RM569156) **Disciplina:** Prompt and Artificial Intelligence — FIAP × GoodWe Brasil — 2026.2

### 1. Resumo da evolução

Nas Sprints 1 e 2, o núcleo do chatbot era um **loop manual em Python** ( `chatbot.py` ) sobre a API Groq (Llama 3.3 70B): histórico de mensagens mantido como lista de dicts, function calling controlado por um `for` com limite de rodadas, e prompting calibrado por few-shot embutido diretamente no código ( `system_prompt.py` ).

Na Sprint 03, esse núcleo foi **reconstruído em LangChain LCEL**, conforme o Módulo 1 (Aulas 01–04):

| Camada                                                   | Sprint 1/2 (legado)                                                                                   | Sprint 03 (LCEL)                                                                                                                                                                                                     |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Orquestração do LLM                                      | Chamadas diretas ao SDK <code>groq</code> , loop manual de tool-calling em <code>chatbot.py</code>    | Chain declarativa <code>prompt \l</code><br><code>llm.with_structured_output(...)</code> , composta em <code>src/chain/builder.py</code>                                                                             |
| Memória                                                  | Lista de mensagens truncada por <b>contagem de mensagens</b> ( <code>MAX_HISTORY_MESSAGES=20</code> ) | <code>RunnableWithMessageHistory</code> + <code>TokenBufferChatMessageHistory</code> (inspirada no <code>ConversationTokenBufferMemory</code> ), truncada por <b>contagem de tokens</b>                              |
| Dados concretos (consumo, status, agendamento, potência) | <b>Function calling</b> : o LLM decidia se e quando chamar uma tool de <code>tools.py</code>          | <b>Structured output</b> ( <code>ConsultaRecarga</code> , Pydantic v2) classifica a intenção; o <b>Python</b> decide deterministicamente qual lookup fazer em <code>src/data/mock_data.py</code>                     |
| Resposta final                                           | Texto livre ( <code>message.content</code> )                                                          | Objeto validado <code>RespostaChat</code> (Pydantic v2), com <code>mensagem</code> , <code>precisa_escalar</code> , <code>destino_escalada</code> , <code>fonte_dados</code>                                         |
| System prompt                                            | Um único arquivo <code>system_prompt.py</code> , sem versionamento explícito                          | Versionado em <code>prompts/</code> ( <code>system_prompt_v1.md</code> = baseline, <code>system_prompt_v2.md</code> = XML tagging), com tabela de mudanças em <code>prompts/VERS0ES.md</code>                        |
| Guardrails                                               | Inteiramente delegados ao próprio LLM via instrução no prompt                                         | Duas camadas: instrução no prompt + checagem determinística em Python ( <code>src/guardrails/</code> ), incluindo o guardrail <b>nov</b> o de domínio sensível (jurídico/financeiro/elétrico) exigido pela Sprint 03 |
| Multi-provider                                           | Só um modelo (llama-3.3-70b via Groq)                                                                 | <code>src/chain/llm_factory.py</code> permite trocar entre <code>openai/gpt-oss-120b</code> e <code>openai/gpt-oss-20b</code> , ambos hospedados na Groq (mesma API key), por variável de ambiente                   |

O código legado ( `chatbot.py` , `tools.py` , `system_prompt.py` , `modelo_de_testes.py` ) foi **mantido intacto** no repositório, para servir de baseline real do comparativo antes/depois (seção 3) — não foi reescrito nem removido.

### 2. Refatoração — decisões técnicas e trade-offs

**2.1. Function calling -> extração estruturada + lookup determinístico.** Na Sprint 2, o modelo decidia livremente se chamava uma tool. Isso funcionou na maioria dos casos, mas falhou de forma documentada no **Teste 5** da Sprint 2 (ver seção 4, Problema 1): o modelo respondeu "não consegui consultar a orquestração de potência" em vez de acionar `consultar_orquestracao_potencia()`. Na Sprint 03, a extração de intenção ( `ConsultaRecarga` , via `llm.with_structured_output` ) é **sempre** executada quando a pergunta não é bloqueada por guardrail, e o lookup do dado mockado correspondente é disparado pelo **Python**, não mais como uma decisão opcional do LLM. Trade-off: isso custa uma chamada de LLM a mais por turno (extração + resposta final = 2 chamadas, contra 1 a 2 na Sprint 2 dependendo do tool-calling), mas elimina uma classe inteira de falha por omissão de tool call.

**2.2. Memória por mensagens -> memória por tokens.** `MAX_HISTORY_MESSAGES=20` (Sprint 2) trata "mensagem curta de 1 linha" e "mensagem longa com lista numerada de 5 passos" como equivalentes para fins de limite de contexto — o que é impreciso. `TokenBufferChatMessageHistory` (Sprint 03) poda o histórico por tokens reais, mais alinhado ao real gargalo (tamanho da janela de contexto do modelo).

**2.3. Por que não usar `ConversationTokenBufferMemory` diretamente.** A classe legada `langchain.memory.ConversationTokenBufferMemory` não implementa a interface `BaseChatMessageHistory` exigida por `RunnableWithMessageHistory` (a API de memória recomendada para LCEL). Optamos por reimplementar o comportamento de poda por tokens dessa classe em `TokenBufferChatMessageHistory`, compatível com `BaseChatMessageHistory`, em vez de usar duas APIs de memória

conflitantes ao mesmo tempo. Essa decisão é documentada explicitamente no código ( `src/chain/memoria.py` ) para deixar clara a diferença em relação ao nome citado no enunciado.

**2.4. Prompt em texto corrido -> XML tagging.** A versão v2 do system prompt organiza cada seção em tags XML ( `<regras_de_comportamento>` , `<formato_de_saida>` etc.), o que: (a) reduz ambiguidade de onde uma seção começa/termina para o modelo, prática de context engineering da Aula 04; e (b) permitiu remover redundância de texto entre seções, reduzindo o prompt em **14,2% dos caracteres** (medição exata — ver `prompts/VERSOES.md` ).

**2.5. Trade-off aceito: duas chamadas de LLM por turno.** A separação extração/resposta final aumenta a latência e o custo em tokens por turno frente à Sprint 2 nos casos em que a Sprint 2 conseguia resolver em uma única chamada sem tool call (ex.: pergunta puramente instrucional, como "como agendar?"). Como os dois modelos da Sprint 03 também são acessados via API da Groq (mesma infraestrutura da Sprint 2 — ver Problema 4), esse trade-off tem um custo real e mensurável em tokens de API, não apenas teórico. Julgamos aceitável porque o ganho em confiabilidade do lookup de dados é o requisito mais crítico do domínio (não inventar números de consumo/potência): preferimos gastar mais tokens por turno a arriscar uma resposta que inventa dados de consumo ou potência.

**2.6. Validação estática realizada (sem chamadas reais ao LLM).** Mesmo sem acesso à API da Groq, foi possível validar estruturalmente todo o código antes da entrega: os dois schemas Pydantic v2 (incluindo o `model_validator` de `RespostaChat` , que corretamente rejeita `precisa_escalar=True` sem `destino_escalada` ), os dois guardrails determinísticos, os dados mockados, o carregamento dos dois prompts versionados, a montagem dos `ChatPromptTemplate` e a construção completa da chain ( `construir_pipeline` , `construir_chatbot_com_memoria` ) foram todos exercitados com sucesso via testes automatizados de sanidade. Os dois caminhos de guardrail (jailbreak e domínio sensível) foram confirmados retornando a resposta correta **sem sequer chamar o LLM**, como esperado pelo design em "defesa em profundidade" descrito na seção 2.1. Um achado real desse teste: o LangChain emite `LangChainDeprecationWarning` ao instanciar `RunnableWithMessageHistory` , recomendando `LangGraph` para persistência de memória em versões futuras — API explicitamente fora do escopo desta Sprint (ver enunciado, "NÃO OBRIGATÓRIO"), então o uso de `RunnableWithMessageHistory` foi mantido conscientemente, mas vale registrar para quem for evoluir o projeto no Módulo 3.

### 3. Tabela de comparativo antes/depois (evidência do refactory)

**Nota de proveniência dos dados.** A coluna "Sprint 1/2 (legado)" vem de `resultados_testes.md` (execução original, 14/06/2026). A coluna "Sprint 03 (LCEL)" vem da execução real feita pela equipe em 18/09/2026 ( `python evals/run_eval.py --mode sprint3 --provider groq_oss e --provider groq_secundario` ), registrada por completo em `evals/sprint3_results.json` . A avaliação qualitativa por caso (adequada/inadequada) da Sprint 03 foi feita comparando cada resposta com o `criterio` do `eval_set.json` ; a mesma avaliação da Sprint 2 nunca tinha sido preenchida no `resultados_testes.md` original (o campo ficou como placeholder "preencher manualmente"), então a linha abaixo usa o único dado qualitativo que o próprio arquivo original registrou em texto livre (a falha do Teste 5). A contagem exata de tokens do prompt (via `tiktoken` ) ainda depende de `python evals/measure_tokens.py` rodado localmente — mantido como estimativa char/4 aqui.

| Métrica                                                  | Sprint 1/2 (legado, Groq/llama-3.3-70b)                                                                                                                                         | Sprint 03 — gpt-oss-120b (padrão)                                                                                                                                                   | Sprint 03 — gpt-oss-20b (bônus)                                                                                 |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Qualidade das respostas (nota manual do eval)            | 6/7 casos originais responderam de acordo com o critério; <b>Teste 5 falhou</b> (não acionou a tool de orquestração de potência); avaliação formal nunca preenchida pela equipe | <b>8/9 adequadas</b> — falhou o caso 1 (relatou "não encontrado" para um consumo que existe nos dados mockados; falha de extração da unidade, não do lookup em si — ver Problema 9) | <b>9/9 adequadas</b> — nenhuma falha qualitativa nos 9 casos                                                    |
| Caso da orquestração de potência (id=5)                  | <b>Falhou</b> (Problema 1: function calling não foi acionado)                                                                                                                   | <b>Corrigido</b> — dados reais corretos (22,0/14,0/8,0 kW), 6,39s                                                                                                                   | <b>Corrigido</b> — dados reais corretos, 21,78s                                                                 |
| Tokens por turno (entrada da pergunta, média)            | Ex. Teste 1: 22 tokens (estimativa char/4)                                                                                                                                      | 27,6 tokens (média dos 9 casos)                                                                                                                                                     | 27,6 tokens (mesmas perguntas)                                                                                  |
| Tokens por turno (saída da resposta, média)              | Ex. Teste 1: 66 tokens (estimativa char/4)                                                                                                                                      | 114,8 tokens                                                                                                                                                                        | 94,8 tokens                                                                                                     |
| Latência média por turno                                 | <b>Não instrumentada</b> na Sprint 2                                                                                                                                            | 6,46s (1,87s se excluídos 2 outliers de ~22-24s, casos 3 e 4)                                                                                                                       | 10,86s (4,76s se excluído 1 outlier de 59,7s, caso 4 — ver abaixo)                                              |
| Acurácia do structured output (sem precisar de fallback) | N/A (Sprint 2 usava function calling, não schema Pydantic)                                                                                                                      | <b>9/9 (100%)</b> — nenhuma chamada precisou do fallback de texto livre                                                                                                             | <b>8/9 (88,9%)</b> — caso 4 caiu no fallback após 2 tentativas ( <code>tool_use_failed</code> , ver Problema 8) |
| Tamanho do system prompt                                 | 6.854 caracteres / 982 palavras (v1)                                                                                                                                            | 5.884 caracteres / 796 palavras (v2) — <b>-14,2% caracteres</b> (medição exata, ver <code>prompts/VERSOES.md</code> )                                                               | (mesmo prompt v2 para os dois modelos)                                                                          |

**Leitura dos resultados (análise real, não hipótese):** - O caso que **falhava na Sprint 2** (orquestração de potência) agora funciona nos dois modelos da Sprint 03 — evidência direta de que a extração estruturada + lookup determinístico (Problema 1) resolveu o problema de raiz. - Curiosamente, o modelo **maior (gpt-oss-120b)**, que é o padrão exigido pelo enunciado, teve uma falha qualitativa que o modelo **menor (gpt-oss-20b)** não teve (caso 1): a extração de intenção não capturou corretamente a unidade "101" da pergunta, fazendo o lookup retornar "não encontrado" mesmo com o dado existindo. Isso mostra que o refactory para structured output reduz mas não elimina 100% o risco de erro de extração — ver Problema 9 para a análise completa. - Em compensação, o modelo maior teve **100% de sucesso no structured output sem fallback**, enquanto o menor precisou do fallback de texto livre uma vez (caso 4), com uma latência bem mais alta nesse caso específico (59,7s, por causa das tentativas + backoff antes de cair no fallback) — isso infla a média de latência do modelo menor de forma enganosa; sem esse outlier, o modelo menor foi na verdade mais rápido em média (4,76s vs. 1,87s do modelo maior, também sem outliers). - As métricas de latência têm um limite de medição importante: `run_eval.py` mede apenas o tempo total do turno (que inclui as 2 chamadas de LLM: extração + resposta final), mas os tokens de entrada/saída registrados são só os da pergunta do usuário e da

## 4. Problemas encontrados e soluções

**Problema 1 — Function calling "fantasma" na orquestração de potência (Sprint 2).** No `resultados_testes.md` original da Sprint 2, o Teste 5 (síndico perguntando sobre orquestração de potência) recebeu a resposta: *"Infelizmente, não consegui consultar a orquestração de potência no momento. Você pode verificar o status atual no painel administrativo..."* — ou seja, o modelo **não acionou** `consultar_orquestracao_potencia()`, apesar de a ferramenta estar disponível e de os outros casos de function calling (Testes 1 e 3) terem funcionado corretamente na mesma execução.

**Decisão tomada:** na Sprint 03, a extração de intenção (`ConsultaRecarga.tipo_consulta`) deixa de ser uma tool call opcional e passa a ser a **saída obrigatória** de uma chamada de LLM com `with_structured_output`, validada por schema Pydantic v2; o lookup de dados correspondente é então disparado deterministicamente pelo código Python (`_LOOKUP_POR_TIPO` em `src/chain/builder.py`), não mais como decisão do modelo. Isso elimina por construção a possibilidade de o modelo "esquecer" de consultar o dado.

**Problema 2 — `ConversationTokenBufferMemory` não é compatível com `RunnableWithMessageHistory`.** O enunciado da Sprint 03 cita explicitamente `RunnableWithMessageHistory` + `ConversationTokenBufferMemory` como a combinação esperada para memória com limite de tokens. Ao planejar a implementação, identificamos que `ConversationTokenBufferMemory` (API de memória legada do LangChain) não implementa a interface `BaseChatMessageHistory` que `RunnableWithMessageHistory` espera receber da função de histórico por sessão — são duas gerações diferentes da API de memória do LangChain. **Decisão tomada:** reimplementar o comportamento de poda por tokens dessa classe em `TokenBufferChatMessageHistory` (`src/chain/memoria.py`), uma subclasse de `BaseChatMessageHistory` que replica a lógica essencial (somar tokens das mensagens, descartar as mais antigas ao ultrapassar o limite), documentando essa decisão no próprio código para deixar rastreável a diferença em relação ao nome citado no enunciado.

**Problema 3 — Ambiente de desenvolvimento sem acesso à API da Groq durante a montagem deste pacote de arquivos.** Ao gerar os arquivos desta Sprint com apoio de IA, o ambiente de execução usado não tinha acesso de rede a `api.groq.com`, o que impediu testar a chain de ponta a ponta e gerar números reais de latência/qualidade para a tabela da seção 3. **Decisão tomada:** todo o código foi escrito e revisado estaticamente (imports, assinaturas de API do LangChain, consistência entre schemas e chains), o eval set foi reaproveitado fielmente da Sprint 2, e os scripts de medição (`run_eval.py`, `measure_tokens.py`) foram entregues prontos para execução local pela equipe — com a tabela da seção 3 deixando explícito o que é medição real (Sprint 2, extraída do `resultados_testes.md` original) e o que é pendente de execução local (Sprint 03).

**Problema 4 — Equipe sem hardware local para rodar gpt-oss-120b via Ollama.** O enunciado da Sprint 03 especifica `ChatOllama` (`gpt-oss:120b`) como o LLM da chain principal (Aula 01). Rodar um modelo de 120B parâmetros localmente exige GPU/RAM que a equipe não possui. **Decisão tomada:** os dois modelos comparados são acessados via **API da Groq (GroqCloud)**, que hospeda nativamente `openai/gpt-oss-120b` — mesma `GROQ_API_KEY`, mesmo SDK (`langchain-groq`), apenas o parâmetro `model` muda entre `"groq_oss"` e o segundo modelo em `src/chain/llm_factory.py`. Essa decisão preserva o requisito funcional do enunciado (a chain roda de fato sobre o `gpt-oss-120b`) sem exigir infraestrutura local que a equipe não tem. O suporte a Ollama local foi mantido no código como caminho opcional (`provider="ollama"`), documentado mas não usado por padrão, para o caso de algum integrante ter acesso a hardware compatível no futuro.

**Problema 5 — O segundo modelo planejado (`llama-3.3-70b-versatile`) foi descontinuado pela Groq durante o desenvolvimento.** Ao rodar `evals/run_eval.py --mode sprint3 --provider groq_ollama` pela primeira vez, a equipe recebeu `Error code: 404 - model_not_found: The model 'llama-3.3-70b-versatile' does not exist or you do not have access to it`. Investigação confirmou que a Groq **descontinuou oficialmente esse modelo em 16/08/2026** (anunciado em 17/06/2026), recomendando como substitutos `openai/gpt-oss-20b` ou `qwen/qwen3.6-27b`. Isso afeta não só a Sprint 03: o `chatbot.py` da Sprint 2 (mantido intocado como baseline "legado") também usa `llama-3.3-70b-versatile` e, por consequência, **também parou de rodar ao vivo** — não por um bug do projeto, mas por uma mudança externa da Groq posterior à entrega da Sprint 2. Os dados históricos da Sprint 2 já capturados em `resultados_testes.md/evals/sprint3_results.json` (antes da descontinuação) continuam válidos como evidência para a tabela da seção 3 — não é necessário reexecutar o legado para manter esse comparativo.

**Problema 6 — Rate limit (429) do tier gratuito da Groq ao rodar o eval set.** Na primeira execução completa, `run_eval.py --mode sprint3 --provider groq_oss` travou aparentemente (na verdade estava em backoff automático do SDK após um `429 Too Many Requests`), levando a equipe a interromper com Ctrl+C. O tier gratuito da Groq limita a cerca de 30 requisições/minuto por modelo, e cada caso do eval faz 2 chamadas de LLM (extração + resposta final) — 9 casos × 2 chamadas facilmente aproximam desse limite, especialmente rodando os dois modelos em sequência. **Decisão tomada:** (1) reduzir `max_retries` do ChatGroq (de `src/chain/llm_factory.py`) para falhar mais rápido em vez de ficar em backoff longo e silencioso; (2) adicionar uma pausa de alguns segundos entre casos em `run_eval.py`; (3) fazer o script salvar os resultados parciais já obtidos mesmo se interrompido por Ctrl+C, para que uma interrupção não jogue fora o progresso já feito.

**Problema 7 — O substituto recomendado (`qwen/qwen3.6-27b`) também retornou "model not found" na conta da equipe.** Ao tentar `LLM_PROVIDER=groq_qwen`, a equipe recebeu o mesmo tipo de erro 404, mesmo `qwen/qwen3.6-27b` sendo um modelo documentado publicamente pela Groq (inclusive como substituto oficial recomendado para o `llama-3.3-70b-versatile` descontinuado — Problema 5). A causa mais provável é que modelos recém-lançados na Groq às vezes ficam em acesso restrito/preview por conta ou tier, mesmo já documentados publicamente — "existe no catálogo" e "sua chave tem acesso" são coisas diferentes. **Decisão tomada:** (1) trocar o segundo modelo padrão para `openai/gpt-oss-20b`, que está no catálogo padrão (não preview) da Groq, com rate limits publicamente listados, reduzindo o risco de mais um 404; (2) tornar o nome do modelo secundário configurável via `GROQ_MODEL_SECUNDARIO` no `.env` (provider renomeado para `groq_secundario`), para que trocar de modelo não exija mais editar código-fonte; (3) documentar no próprio `llm_factory.py` o comando `curl https://api.groq.com/openai/v1/models -H "Authorization: Bearer $GROQ_API_KEY"` para a equipe confirmar, a partir da própria conta, quais modelos estão realmente acessíveis antes de escolher o segundo modelo do comparativo — em vez de confiar apenas na documentação pública, que nem sempre reflete o acesso liberado por conta/tier.

**Problema 8 — `tool_use_failed` (erro 400) em casos benignos, e um bug real no guardrail jurídico exposto no processo.** Ao rodar o eval set completo com `groq_secundario` (`openai/gpt-oss-20b`), os casos 2 ("como agendar"), 3 (diagnóstico do box 7) e 9 (guardrail jurídico) retornaram `Error code: 400 - tool_use_failed: Tool choice is required, but model did not call a tool`, com `failed_generation: "Desculpe, mas não posso ajudar com essa solicitação."`. Investigação revelou duas causas distintas, sobrepostas no mesmo sintoma: (a) para os casos 2 e 3 (perguntas totalmente benignas), é um problema conhecido de modelos de raciocínio forçados a `tool_choice` obrigatório: o modelo às vezes prefere responder em texto livre, e a Groq trata isso como erro em vez de aceitar a resposta em texto; (b)

para o caso 9, o texto da pergunta do eval ("...posso processar **ele** na justiça") não batia com o regex do guardrail jurídico (`processar (o|a) síndico`, que exigia as palavras adjacentes), então a pergunta chegou ao LLM sem ser bloqueada — e o próprio modelo, por conta própria, se recusou a responder um pedido de aconselhamento jurídico, só que em formato incompatível com o `tool_choice` forçado. **Decisão tomada:** (1) corrigir o regex de `src/guardrails/moderation.py` para cobrir a variação "processar ele/ela ... justiça", não só "processar o/a síndico" adjacente — o caso 9 agora é bloqueado deterministicamente antes de qualquer chamada ao LLM, como deveria desde o início; (2) adicionar uma chain de fallback sem `tool_choice` forçado (`final_chain_bruta`, texto livre) em `src/chain/builder.py`, usada automaticamente quando a chamada estruturada falha mesmo após uma segunda tentativa — o usuário recebe uma resposta (estruturada ou em texto simples) em vez de um erro 500/exceção não tratada. Esse problema reforça um princípio geral: guardrails determinísticos (regex) sempre devem ser a primeira linha de defesa quando possível, porque são mais previsíveis do que depender do modelo se recusar "corretamente" em todo formato de saída.

**Problema 9 — O modelo padrão (gpt-oss-120b) extraiu incorretamente a unidade em um caso, mesmo com o dado explícito na pergunta.** Na execução real do eval set (18/09/2026), o caso 1 ("Sou o morador do apartamento 101. Quanto eu consumi...") retornou, com `gpt-oss-120b`, a resposta "não encontrou registros de consumo para o apartamento 101" — apesar de `mock_data.py` ter o registro real (42,5 kWh / R\$ 38,25). O mesmo caso, com `gpt-oss-20b`, retornou a resposta correta. Como o lookup em `src/data/mock_data.py` é determinístico (não é o LLM que decide os números, só qual unidade consultar), a causa mais provável é que a extração de intenção (`ConsultaRecarga.unidade`) não capturou "101" corretamente para esse modelo nessa chamada específica — um tipo de falha que o refactory da Sprint 03 **reduz** (ao tornar o lookup determinístico, como fez com o Problema 1), mas **não elimina 100%**, porque a extração da unidade ainda depende do LLM entender a pergunta corretamente. **Decisão tomada:** documentar o caso como está (sem "consertar" escondendo o resultado real), porque é uma evidência genuína e relevante do comparativo de modelos — contraintuitivamente, o modelo maior (e mais caro) errou onde o menor acertou nesse caso específico. Como melhoria futura (fora do escopo desta Sprint, registrada aqui para o Módulo 3), vale considerar: (a) few-shot examples adicionais focados especificamente em extração de unidade a partir de frases com "apartamento N"/"unidade N"; (b) um `field_validator` em `ConsultaRecarga.unidade` que tente extrair um padrão numérico via regex da pergunta original como camada de verificação, alertando (sem sobrescrever automaticamente) quando o valor extraído pelo LLM diverge de um número claramente presente no texto.

## 5. Equipe e divisão de trabalho

Os nomes e RMs abaixo são os mesmos registrados no `README.md` do repositório desde a Sprint 1/2. A divisão de tarefas por pessoa foi confirmada pela equipe.

| Integrante        | RM     | Tarefa principal                        |
|-------------------|--------|-----------------------------------------|
| André Fujinaga    | 569158 | Chain LCEL + Multi-provider             |
| Arthur Machado    | 569919 | Memória conversacional                  |
| Conrado Gracie    | 569157 | Structured Output (Pydantic v2)         |
| Guilherme Belo    | 570079 | Context Engineering + Prompt Versionado |
| Renato Sandreschi | 569156 | Guardrails + Eval/Relatório             |

Relatório gerado como parte da entrega da Sprint 03. Ver também `docs/relatorio_modelos.md` para o comparativo de modelos/parâmetros (bônus multi-provider) e `prompts/VERSOES.md` para o detalhamento do versionamento do system prompt.